

Strategie Di Distribuzione

- Le classi di un'applicazione, es. Clienti, Ordini, Prodotti, etc. potrebbero essere distribuite, in modo che ciascun oggetto sia posizionato su un suo host
 - Una chiamata di metodo da un processo su un host ad uno su un altro host è molto più lenta di una chiamata interna allo stesso processo
 - Circa 20.000 ns contro 1 ns (in dipendenza di distanze e tecnologie)
- Poiché le chiamate remote sono lente, è meglio combinare in una chiamata remota varie operazioni (es. aggiornamento di tanti dati)
- L'interfaccia per un oggetto remoto è differente rispetto a quella per un oggetto locale
 - Un oggetto locale ha un'interfaccia fine-grained, segue il principio generale OO di combinare piccole parti e rifinirle in vari modi per estendere la progettazione
 - L'interfaccia per un oggetto remoto è coarse-grained

1

Prof. Tramontana - Novembre 2022

Strategie Di Distribuzione

- La programmazione diventa più difficile con interfacce coarse-grained
- Se la strategia di distribuzione è basata sulle classi, ci saranno tante chiamate remote, interfacce non appropriate (per cercare di limitare le tante chiamate), e un sistema difficile da modificare
- Come distribuire quindi efficacemente?
 - Inserire le classi in un singolo processo e replicare il processo su più host. Localmente allo stesso processo si hanno interfacce fine-grained
 - Minimizzare i confini della distribuzione e usare gli host per raggruppamenti di classi

2

Prof. Tramontana - Novembre 2022

Dove Separare

- Un sistema client e server, in cui i pc desktop condividono i dati presenti su un repository, suggerisce una separazione fra processi
- Una seconda separazione è possibile fra la parte server ed il database
 - Le stored procedures, permettono di eseguire la parte server sul processo del database, ma non è pratico
- Un'altra divisione può esserci fra il web server e l'application server. In alcuni casi questi due sono in un singolo processo
- Package forniti da diversi fornitori potrebbero eseguire in processi diversi
- Infine, se si hanno buoni motivi si può separare la parte server dell'applicazione in più processi

3

Prof. Tramontana - Novembre 2022

Come Separare

- Quando si progetta un sistema software si devono limitare i confini di distribuzione il più possibile
- Per progettare un sistema distribuito usando oggetti fine-grained bisogna
 - Usare oggetti fine-grained internamente allo stesso host
 - Mettere oggetti coarse-grained ai confini della distribuzione, il cui solo scopo è fornire un'interfaccia remota ad oggetti fine-grained
 - Gli oggetti coarse-grained agiscono da *Facade* per oggetti fine-grained. I *Remote Facade* minimizzano le difficoltà create da interfacce coarse-grained
 - Per trasferire coarse-grained object usare i *Data Transfer Object*

4

Prof. Tramontana - Novembre 2022

Design Pattern

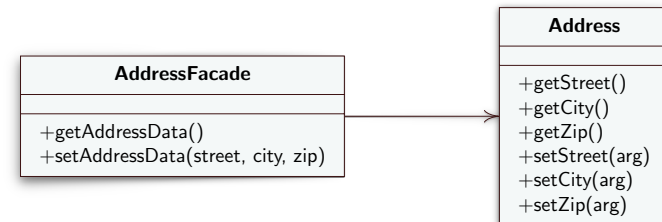
Remote Facade

5

Prof. Tramontana - Novembre 2022

Remote Facade

- **Intento:** Fornisce una facciata coarse-grained su oggetti fine-grained per migliorare l'efficienza sulla rete



6

Prof. Tramontana - Novembre 2022

Remote Facade

- Problema
 - Le chiamate inter-processo sono molto più costose delle chiamate interne al processo, persino sulla stessa macchina
 - La lentezza è dovuta a: scambio dati, check di sicurezza, pacchetti che viaggiano fra switch, latenza di rete
 - Oggetti remoti che presentano interfacce coarse-grained permettono di minimizzare le chiamate ma complicano gli oggetti
 - L'effetto sul codice è una minore chiarezza di implementazione (l'intenzione della chiamata si perde, il codice si complica per ricevere o trasmettere più dati, etc.), e minore controllo
 - La programmazione diventa più difficile e la produttività rallenta

7

Prof. Tramontana - Novembre 2022

Remote Facade

- Soluzione
 - Un Remote Facade offre una facciata coarse-grained per oggetti fine-grained
 - Nessuno degli oggetti fine-grained ha un'interfaccia remota ed il Remote Facade non contiene logica di dominio
 - Il Remote Facade traduce metodi coarse-grained in oggetti sottostanti fine-grained
 - Il Remote Facade affronta la distribuzione separando responsabilità distinte in oggetti distinti (in accordo a OO)

8

Prof. Tramontana - Novembre 2022

Remote Facade

- Come funziona
 - La logica complessa è inserita in oggetti piccoli che sono progettati per collaborare all'interno dello stesso processo. Per permettere un accesso remoto efficiente ad essi, si introduce un facade che agisce come interfaccia remota
 - Tipicamente, un Remote Facade sostituisce i vari metodi get e set con un solo metodo get e un solo metodo set, chiamati bulk accessors
 - Quando un client chiama un bulk setter, il facade legge i dati e chiama i singoli set sul vero oggetto
 - Un Remote Facade può fare da facciata a più oggetti piccoli

9

Prof. Tramontana - Novembre 2022

Remote Facade

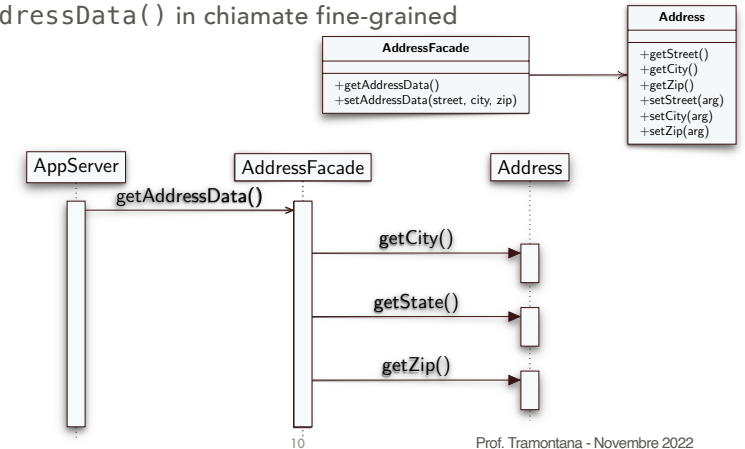
- Per trasferire informazioni in grandi quantità occorre avere altri oggetti a supporto
 - Se il tipo restituito da `getAddressData()` è noto ad entrambi i lati della connessione ed è serializzabile, il metodo `getAddressData()` crea una copia dell'oggetto originale
 - Se questo non si può fare, poiché gli oggetti non sono noti al client, o non si vuole inviare tutto l'oggetto, si ricorre al *Data Transfer Object*
- Si progettano i Remote Facade in base alle necessità del client
 - Si hanno pochi Remote Facade per un'applicazione, ogni Remote Facade ha tanti metodi a servizio di varie versioni di client (remoti)
 - Il Remote Facade espone un metodo che facilita il client, e più metodi del facade possono avviare lo stesso metodo interno

11

Prof. Tramontana - Novembre 2022

Remote Facade

- Un oggetto Address ha metodi fine-grained
- Un oggetto AddressFacade converte la chiamata getAddressData() in chiamate fine-grained



Prof. Tramontana - Novembre 2022

Remote Facade

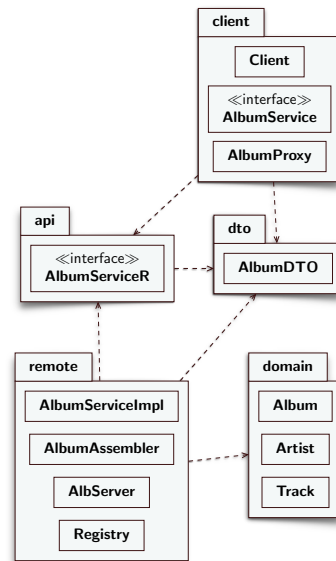
- Varie responsabilità possono essere aggiunte al Remote Facade
- I suoi metodi sono un punto dove poter avviare **controlli di sicurezza**. Una access control list può dire quali utenti possono fare chiamate su quali metodi
- Si possono applicare controlli di gestione per le **transazioni**. Un metodo del facade può avviare la transazione, fare il lavoro interno, quindi fare **commit alla fine**. Ogni chiamata è una indicazione di una transazione, poiché non si vuol lasciare la transazione aperta quando si ritorna al client (tempi lunghi e imprevisti dovuti alla rete)
- Il Remote Facade non dovrebbe contenere logica del dominio

12

Prof. Tramontana - Novembre 2022

Esempio Remote Facade

- Il package dto contiene Data Transfer Objects, che aiutano a trasmettere i dati al client tramite rete. Hanno metodi per l'accesso ai dati e possono essere serializzati
- Il package remote contiene oggetti che passano i dati fra oggetti del dominio e Data Transfer Objects
- AlbumServiceR è l'interfaccia del Remote Facade e la sua implementazione è AlbumServiceImpl
- Sull'host client vi sono i package: client, api e dto
- Sull'host server vi sono i package: api, dto, remote e domain



13

Prof. Tramontana - Novembre 2022

Design Pattern Data Transfer Object

14

Prof. Tramontana - Novembre 2022

Data Transfer Object

- Intento: un oggetto trasporta dati fra processi per ridurre il numero di chiamate a metodo
- Problema
 - Quando si lavora con una interfaccia remota, come Remote Facade, ciascuna chiamata è costosa. Per ridurre il numero di chiamate si trasferiscono più dati ad ogni chiamata
 - Si possono usare tanti parametri, ma è scomodo, e il parametro di ritorno è un singolo valore
- Soluzione
 - Creare un Data Transfer Object (DTO) che tiene tutti i dati della chiamata. Deve essere serializzabile per essere trasmesso.
 - Si usa un oggetto assemblatore per trasferire dati fra il DTO e gli oggetti del dominio

15

Prof. Tramontana - Novembre 2022

Data Transfer Object

- Come funziona
 - Un DTO è un oggetto che ha vari campi e metodi getter e setter
 - I campi del DTO sono dati primitivi, classi semplici (String, date), o altri DTO. La struttura fra DTO dovrebbe essere semplice
 - Il DTO porta i dati che l'oggetto remoto ha richiesto. In genere, il DTO contiene più di un singolo oggetto lato server
 - In genere, gli oggetti del dominio sono connessi fra loro e impossibili da serializzare
 - Il DTO è progettato per un particolare client, spesso corrisponde ai dati presenti nelle pagine web
 - Ci possono essere più DTO corrispondenti a richieste diverse, e alle risposte, dipende dai dati in comune che passano (o richiedono) le diverse richieste

16

Prof. Tramontana - Novembre 2022

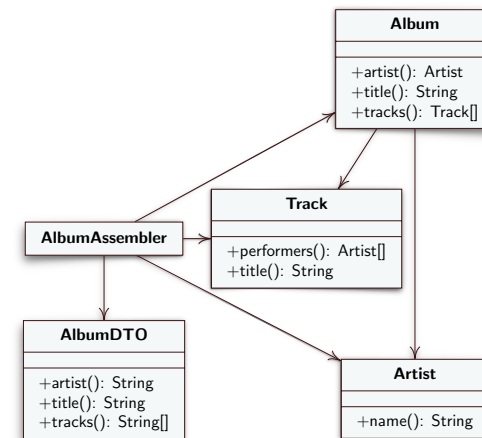
Data Transfer Object

- Oltre ai metodi di accesso, un Data Transfer Object è responsabile per la serializzazione di sé stesso in qualche formato
- Il formato dipende da chi è presente nei due lati della connessione, da cosa può viaggiare in rete, da quanto è facile serializzare
 - Java ha il supporto per la serializzazione in binario
 - Si può serializzare in XML
 - Si può implementare il proprio meccanismo di serializzazione, a partire da una semplice descrizione del record. Si può usare la riflessione computazionale per gestire la serializzazione
- Una serializzazione in binario rende fragile la comunicazione: un aggiornamento del server (che aggiunge o toglie un campo al DTO) che non corrisponde ad un aggiornamento del client, provoca un errore di deserializzazione
- La serializzazione in XML può essere scritta in modo che le classi siano più tolleranti ai cambiamenti

17

Prof. Tramontana - Novembre 2022

Data Transfer Object



18

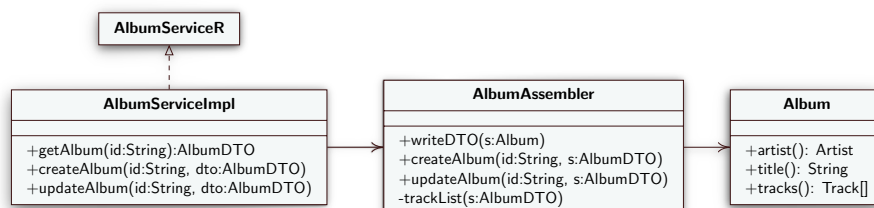
Prof. Tramontana - Novembre 2022

Implementazione Remote Facade

- Il ruolo Remote Facade fornisce una facciata coarse-grained sulla quale arrivano chiamate attraverso la rete, essendo il remote facade AlbumServiceImpl lato server
- A basso livello le chiamate viaggiano sulle socket, come visto per il Remote Proxy, ma l'implementazione è onerosa
- Per l'implementazione usiamo RMI che è un tipo di chiamata a procedura remota indipendente dalla rete e implementata in Java

19

Prof. Tramontana - Novembre 2022



Remote Method Invocation

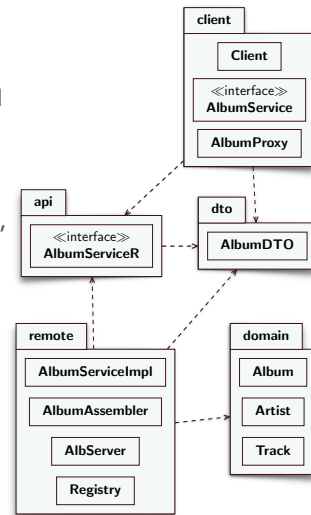
- Per far avvenire la comunicazione fra client e server si definisce una interfaccia Java comune a entrambi i due lati
- Per il codice di esempio del Remote Facade, tale interfaccia è AlbumService, e contiene la definizione dei metodi che vogliamo far usare al client
- Per RMI, l'interfaccia deve essere un sottotipo dell'interfaccia java.rmi.Remote
- Per RMI, per ciascun metodo in AlbumService e AlbumServiceImpl far lanciare un'eccezione
- Lato client
 - Per localizzare un oggetto che esiste in remoto si usa un'operazione di lookup attraverso un identificatore sul servizio di Naming (ovvero RMI registry). Questa operazione fornirà un proxy per accedere all'oggetto remoto
- Lato server
 - Si crea un oggetto, si registra l'oggetto sul servizio di Naming

20

Prof. Tramontana - Novembre 2022

Esempio Di Distribuzione

- AlbumProxy fa sì che il Client non debba gestire le eccezioni necessarie per invocare un metodo remoto con RMI
- AlbumServiceR è l'interfaccia del Remote Facade AlbumServiceImpl (ogni suo metodo può lanciare eccezioni, per via dell'uso di RMI)
- AlbServer crea l'istanza del RemoteFacade e la registra nel registro di RMI per essere trovata quando si fanno chiamate remote



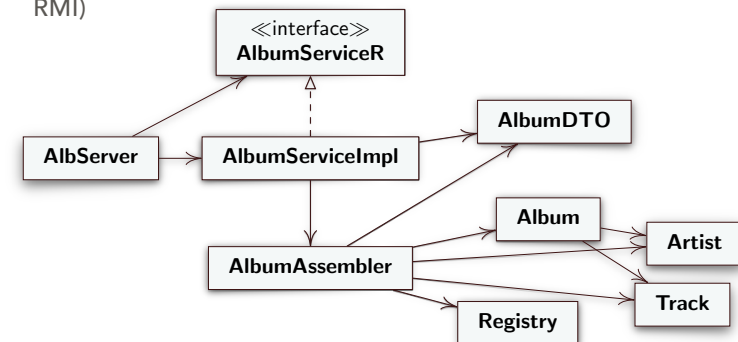
21

Prof. Tramontana - Novembre 2022

Implementazione Remote Facade

Lato Server

- La classe AlbServer contiene il main, crea l'oggetto AlbumServiceImpl e lo registra sul naming service
- AlbumServiceR è l'interfaccia che estende Remote (voluta da RMI)



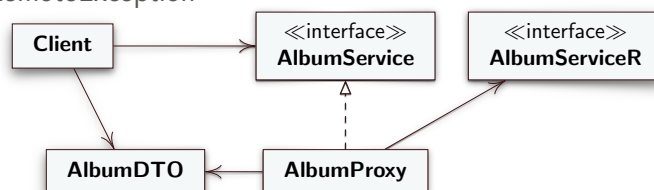
22

Prof. Tramontana - Novembre 2022

Implementazione Remote Facade

Lato Client

- La classe Client contiene il main, chiama metodi su AlbumService
- AlbumProxy conosce la posizione remota del servizio, schermo il client dalla gestione delle eccezioni provocate dalle comunicazioni in rete
- I metodi di AlbumServiceR devono avere throws RemoteException



23

Prof. Tramontana - Novembre 2022

// Remote service implementation (accepting RMI), playing role Remote Facade

```
public class AlbumServiceImpl extends UnicastRemoteObject
    implements AlbumServiceR {

    public AlbumServiceImpl() throws RemoteException {
        super();
    }

    public AlbumDTO getAlbum(String id) throws RemoteException {
        return new AlbumAssembler().writeDTO(Registry.findAlbum(id));
    }

    public void createAlbum(String id, AlbumDTO dto) throws RemoteException {
        new AlbumAssembler().createAlbum(id, dto);
    }

    public void updateAlbum(String id, AlbumDTO dto) throws RemoteException {
        new AlbumAssembler().updateAlbum(id, dto);
    }
}
```

24

Prof. Tramontana - Novembre 2022